

NYCU Introduction to Machine Learning, Homework 3

[112550183], [蔡承志]

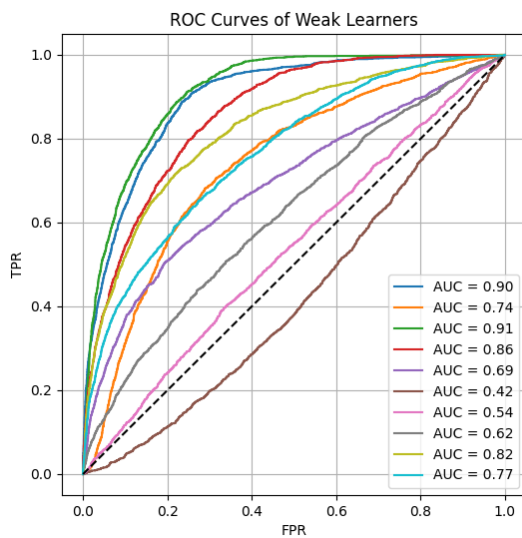
Part. 1, Coding (60%):

(20%) Adaboost

1. (5%) Show your accuracy of the testing data ($n_estimators = 10$)

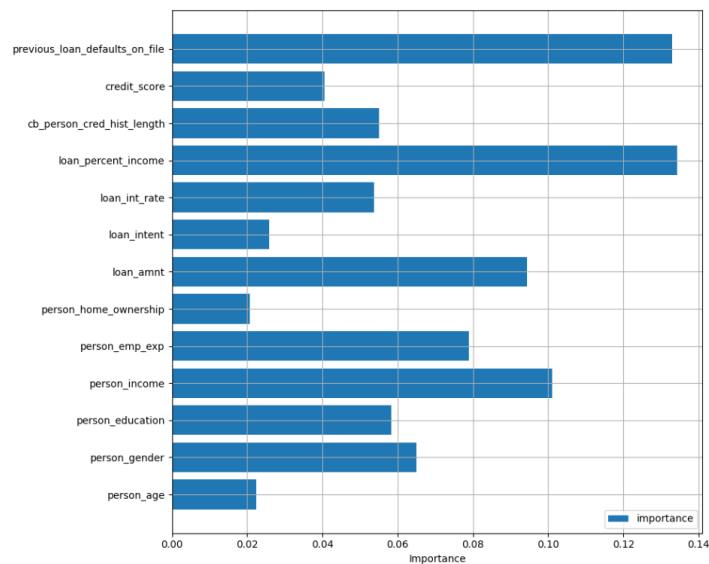
```
2025-11-12 19:45:48.864 | INFO | __main__:main:86 - AdaBoost - Accuracy: 0.8334
# AdaBoost
clf_adaboost = AdaBoostClassifier(input_dim=n_feature, num_learners=10, batch_size=16)
_ = clf_adaboost.fit(X_train, y_train, num_epochs=2, learning_rate=0.01)
```

2. (5%) Plot the AUC curves of each weak classifier.



3. (10%)

- a. Plot the feature importance of the AdaBoost method.



- b. Paste the snapshot of your implementation of the feature importance estimation.

```
def compute_feature_importance(self) -> t.Sequence[float]:
    """
    TODO: Implement the feature importance calculation
    """
    feature_important = []

    for alpha, wc_model in zip(self.alphas, self.learners):
        feature_weights = wc_model.fc.weight.data.abs()
        feature_important.append((feature_weights * alpha).numpy())

    return np.sum(np.array(feature_important), axis=0).reshape(-1)
```

- c. Explain how you compute the feature importance for the Adaboost method shortly (within 100 words as possible)

The weight we get from the weak classifier h_t should multiply with its α_t because it means the power of this weak classifier h_t to influence the result, then add all weak classifier's weight for each feature.

(20%) Bagging

4. (5%) Show the accuracy of the test data using 10 estimators. ($n_estimators=10$)

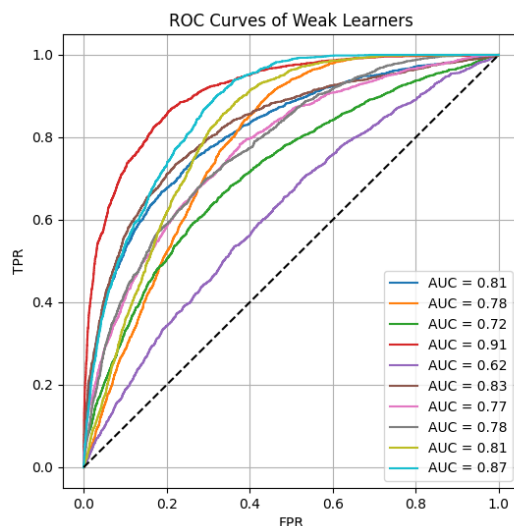
```
2025-11-12 19:45:56.010 | INFO | __main__:main:107 - Bagging - Accuracy: 0.8366
```

```
# Bagging
clf_bagging = BaggingClassifier(input_dim=n_feature)
_ = clf_bagging.fit(X_train, y_train, num_epochs=5, learning_rate=0.005)

# create 10 learners, dont change.
self.learners = [
    WeakClassifier(input_dim=input_dim) for _ in range(10)
]

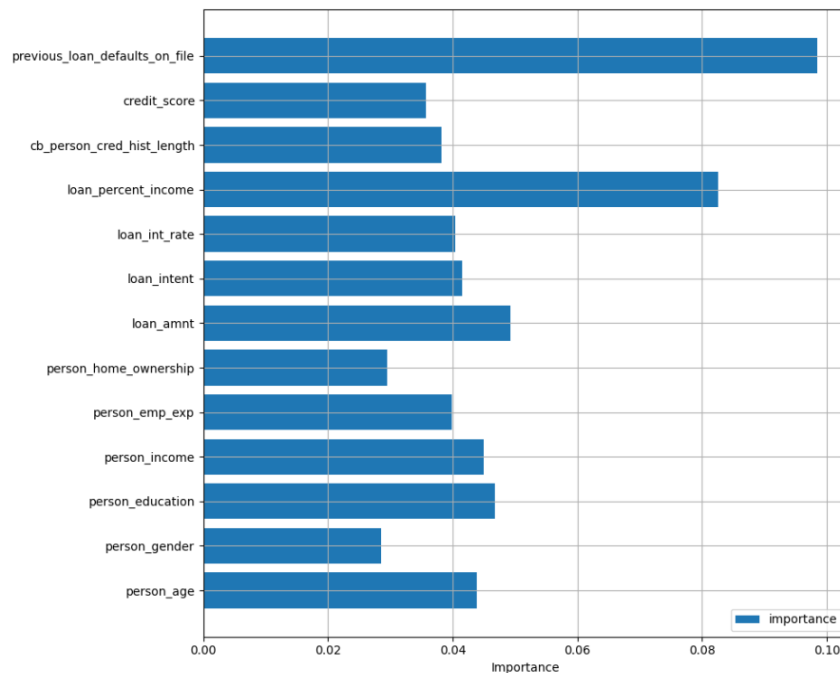
# my
self.batch_size = 128
```

5. (5%) Plot the AUC curves of each weak classifier.



6. (10%)

a. Plot the feature importance of the Bagging method.



b. Paste the snapshot of your implementation of the feature importance estimation.

```
def compute_feature_importance(self) -> t.Sequence[float]:
    """
    TODO: Implement the feature importance calculation
    """
    feature_important = []

    for wc_model in self.learners:
        feature_weights = wc_model.fc.weight.data.abs()
        feature_important.append(feature_weights.numpy())

    return np.sum(np.array(feature_important), axis=0).reshape(-1)
    raise NotImplementedError
```

c. Explain how you compute the feature importance for the Bagging method shortly (within 100 words as possible)

In bagging, all weak classifier h_i has same power, so just sum all weak classifier's weight for each feature is enough (or mean is the same).

(15%) Decision Tree

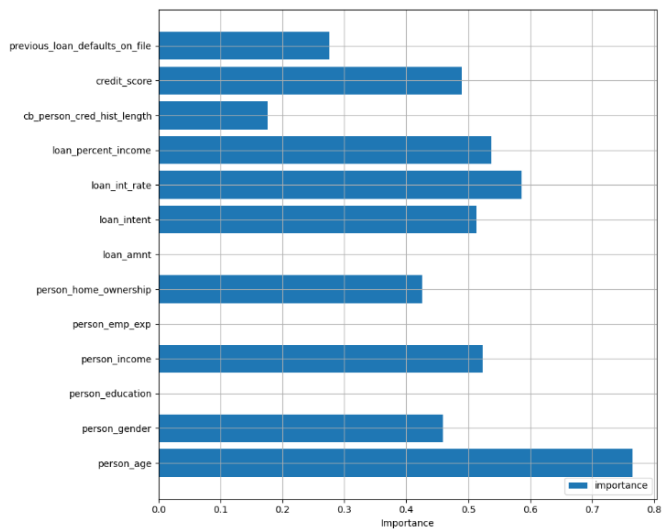
7. (5%) Compute the Gini index and the entropy of the array [0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1].

```
arr: [0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1]
gini_index: 0.49586776859504145
entropy: 0.9940302111884174
```

8. (5%) Show your accuracy of the testing data with a max-depth = 7

```
2025-11-12 19:46:15.298 | INFO | _main :main:137 - DecisionTree - Accuracy: 0.9167
# Decision Tree
clf_tree = DecisionTree(max_depth=7)
```

9. (5%) Plot the feature importance of the decision tree.
Sum the gain if the feature is chosen as the split.



(5%) Code Linting

10. Show the snapshot of the flake8 linting result (paste the execution command even when there is no error).

```
PS C:\Users\percy\OneDrive\桌面\大學資料\3上\ML\MLHW3\114-1-ml-hw3-release> uv run flake8 .\main.py
PS C:\Users\percy\OneDrive\桌面\大學資料\3上\ML\MLHW3\114-1-ml-hw3-release> uv run main.py
```

Part. 2, Questions (40%):

1. (15%)

In the AdaBoost algorithm, each selected weak classifier, h_t is assigned a weight:

$$\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$$

- (a) What is the definition of ϵ_t in this formula?
- (b) If a weak classifier h_t performs only as well as "random guessing" (i.e., $\epsilon_t = 0.5$), what will α_t be?
- (c) Based on your answer in (b), briefly explain how this α_t weight formula ensures the robustness of AdaBoost.

Write or type your answer here.

- (a) Assume there are "m" samples, weak classifier h_t is assign weight on each sample by $D_t()$,

$$\epsilon_t = \sum_{i=1}^m D_t(i) [y_i \neq h_t(x_i)]$$

epsilon can be seen as the total error that weak classifier h_t make under this weight $D_t()$, classifier should try to minimum the ϵ_t by adjust $D_t()$.

- (b) As above question said, so when $\epsilon_t = 0.5$, α_t would closed to 0, means that weak classifier h_t perform similar as "random guessing"; if perform better than "random guessing", α_t get higher; otherwise, get lower α_t .
- (c) As above question said, in final decision of the whole adaboost,

$$pred = sign (H (x) = \sum_{t=1}^T \alpha_t h_t(x))$$

α_t decide each weak classifiers' power to influence the result, perform well weak classifiers can get more power $\alpha_t > 0$ to influence the result, perform similar as "random guessing" $\rightarrow \alpha_t = 0$, ignore them; and due to only 2 classes, the opposite of perform poor can be seen as perform well, $\alpha_t < 0$ poor = $\alpha_t > 0$ well, by this adjustment, we ensure the robustness of Adaboost.

2. (15%) Random Forest is often considered an improvement over Bagging (Bootstrap Aggregating) when used with Decision Trees as the base learners.

(a) Briefly explain the potential problem that can exist among the T classifiers

$(C_1, \dots, C_T)$ produced by Bagging when using decision trees.

(b) To mitigate the problem from (a), Random Forest introduces a second source of randomness in addition to Bagging. Specifically describe what this second source of randomness is and how it works.

Write or type your answer here.

(a) If there is a feature that gets a high weight on prediction, then it will always be the root in every tree, which make trees have "high correlation". This is not what we want because ensemble learning wants to incorporate diverse opinions, this situation limits the model's ability to reduce variance after Bagging.

(b) Random Feature Selection, when checking split, only randomly select feature subset, so if at the time tree split, it doesn't randomly get the high weight feature mentioned above, then this tree should select other feature to split, forces these trees to not always rely on the few "strongest" features.

3. (10%) Is it always possible to find the smallest decision tree that codes a dataset with no error in polynomial time? If not, what algorithm do we usually use to search a decision tree in a reasonable time? Also, list the criterion that we stop splitting the decision tree and leaf nodes are created (List two).

Write or type your answer here.

No, it's not always possible. Finding the smallest decision tree is NP-hard problem.

Using Greedy Algorithm, always select best decision (split) in current situation rather than consider future to get the globally optimal solution, and won't go back. It reduces the trees that need to be search, making computation become possible.

1. All samples on the node have same labels -> Perfectly Pure.
2. Node get the max_depth.